

Scalable Microservices Architecture with ECS and Fargate

Services Used

Amazon ECS
AWS Fargate
Amazon ECR
Amazon RDS MySQL
Amazon CloudWatch
Docker
Docker Compose
React.js
Node.js
Express.js
MySQL

Project Description

This project deploys a scalable microservices-based web application on AWS using Docker containers, Amazon ECS, and AWS Fargate.

The application is divided into separate services:

Frontend Service
Backend API Service
Database Service

The frontend and backend are containerized using Docker and deployed independently using ECS Fargate. Docker images are stored inside Amazon ECR. The backend service connects with Amazon RDS MySQL to store and retrieve data, while Amazon CloudWatch is used for monitoring and log management.

This architecture follows modern cloud-native deployment practices where applications are divided into smaller independent services instead of one large monolithic application.

Definition

What is Microservices Architecture?

Microservices architecture is a software design approach where an application is divided into multiple small independent services.

Each service:

- Runs independently
- Deploys independently
- Can scale independently
- Communicates using APIs

In this project:

- Frontend → React Service
- Backend → Node.js API Service
- Database → RDS MySQL Service

All components are separated and communicate through HTTP APIs.

Why Microservices?

Traditional monolithic applications keep frontend, backend, and database logic inside one application.

Problems with monolithic applications:

- Hard to scale
- Hard to maintain
- Entire application must redeploy
- One failure can affect everything

Microservices solve these problems by separating application components.

Benefits:

- Better scalability
- Better fault isolation
- Independent deployments
- Easy maintenance
- Production-ready architecture

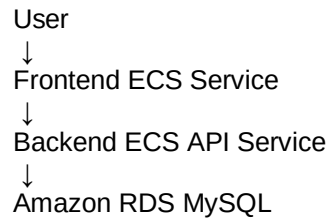
Scope of the Project

This project covers:

- Docker containerization
- Microservices deployment
- AWS ECS orchestration
- Serverless containers using Fargate
- Database integration using RDS
- Cloud monitoring using CloudWatch
- Secure AWS networking
- Security group architecture

The project demonstrates how real-world applications are deployed in cloud production environments.

Architecture Overview



AWS Services Used

1. Amazon ECS

Amazon ECS (Elastic Container Service) is a container orchestration service provided by AWS.

It manages:

- Container deployment
- Container scheduling
- Container scaling
- Container lifecycle

Purpose

Running Docker containers manually becomes difficult when applications grow.

ECS automates: Deployment Scaling Recovery Container management

Role in This Project

ECS runs:

- Frontend container
- Backend container

Each service runs independently as ECS tasks.

2. AWS Fargate

Definition

AWS Fargate is serverless compute for containers.

Normally containers require EC2 servers. Fargate removes the need to manage servers manually.

Purpose

Removes need to manage EC2 servers.

Role in Project

Runs ECS tasks without server management.

3. Amazon ECR

Definition

Amazon ECR (Elastic Container Registry) is a private Docker image registry.

It stores container images securely in AWS.

4. Amazon RDS MySQL

Definition

Amazon RDS is a managed relational database service.

Purpose

Applications require persistent storage.

Instead of storing data locally inside containers, data is stored inside RDS.

5. Amazon CloudWatch

Definition

Monitoring and logging service.

Purpose

Stores application logs.

Role in Project

Used to troubleshoot ECS containers.

Application Details

Frontend Technology

React.js
Vite
JavaScript

Backend Technology

Node.js
Express.js
mysql2
cors
dotenv

Database

MySQL
Database name: microdb
Table: products

Main Features

Microservices architecture
Docker containerization
Serverless deployment using Fargate
Backend API integration
RDS database integration
CloudWatch logging
Scalable infrastructure

Implementation Steps

Step 1: Create Project Structure

Created:
[frontend/](#)
[backend/](#)
[docker-compose.yml](#)
Purpose:
Separate frontend and backend microservices.

```
[root@ip-172-31-10-219 ecs-microservices-project]# ls  
backend  docker-compose.yml  frontend  
[root@ip-172-31-10-219 ecs-microservices-project]#
```

Step 2: Create Backend Service

Created Node.js Express API.
Endpoints:
/api/products
Purpose
Backend handles:
API requests
Database communication
Business logic

Step 3: Create Frontend Service

Created React.js frontend application.
Frontend fetches data from backend APIs.

Purpose:

Frontend only handles UI logic.

Actual data comes from backend APIs.

Step 4: Dockerize Backend

Created backend Dockerfile.

```
[root@ip-172-31-10-219 backend]# ls
Dockerfile  node_modules  package-lock.json  package.json  server.js  server.js.save
[root@ip-172-31-10-219 backend]#
```

Purpose

Converts backend application into portable container image.

Command:

docker build -t backend-service .

Step 5: Dockerize Frontend

Created frontend Dockerfile.

```
[root@ip-172-31-10-219 frontend]# ls
Dockerfile  README.md  eslint.config.js  index.html  node_modules  package-lock.json  package.json  public  src  vite.config.js
[root@ip-172-31-10-219 frontend]#
```

Purpose

Converts frontend into portable Docker container.

Command:

docker build -t frontend-service .

Step 6: Docker Compose Setup

Created docker-compose.yml.

```
[root@ip-172-31-10-219 ecs-microservices-project]# ls
backend  docker-compose.yml  frontend
[root@ip-172-31-10-219 ecs-microservices-project]# cat docker-compose.yml
services:
  backend:
    image: backend-service
    container_name: backend-container
    ports:
      - "5000:5000"

  frontend:
    image: frontend-service
    container_name: frontend-container
    ports:
      - "5173:5173"
    depends_on:
      - backend
[root@ip-172-31-10-219 ecs-microservices-project]#
```

Purpose

Runs frontend and backend together locally.

Command

docker-compose up -d

Step 7: Create Amazon ECR Repositories

Created repositories:

Repository name → frontend-service

Visibility → Private

Tag immutability → Mutable

Encryption → AES-256

Click:

Create repository

Do same again for:

backend-service

Purpose:

Store Docker images in AWS.

Step 8: Push Docker Images to ECR

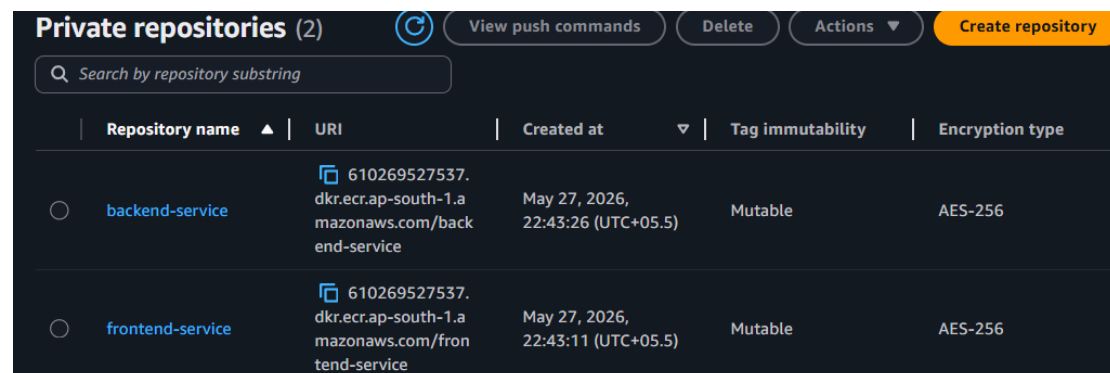
Tagged images:

- Configure aws use acces key and acces ID
- Login ecr in vm
- `aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 610269527537.dkr.ecr.ap-south-1.amazonaws.com`

`docker tag frontend-service:latest ACCOUNT_ID.dkr.ecr.ap-south-1.amazonaws.com/frontend-service:latest`

Pushed images:

`docker push ACCOUNT_ID.dkr.ecr.ap-south-1.amazonaws.com/frontend-service:latest`



The screenshot shows the AWS Private repositories interface. At the top, there's a header with 'Private repositories (2)', a search bar, and buttons for 'View push commands', 'Delete', 'Actions', and 'Create repository'. Below the header is a table with columns: Repository name, URI, Created at, Tag immutability, and Encryption type. Two repositories are listed: 'backend-service' and 'frontend-service'. Both have a URI starting with '610269527537.dkr.ecr.ap-south-1.amazonaws.com/' and were created on May 27, 2026, at 22:43:26 (UTC+05.5). Both are 'Mutable' and use 'AES-256' encryption.

Repository name	URI	Created at	Tag immutability	Encryption type
backend-service	610269527537.dkr.ecr.ap-south-1.amazonaws.com/backend-service	May 27, 2026, 22:43:26 (UTC+05.5)	Mutable	AES-256
frontend-service	610269527537.dkr.ecr.ap-south-1.amazonaws.com/frontend-service	May 27, 2026, 22:43:11 (UTC+05.5)	Mutable	AES-256

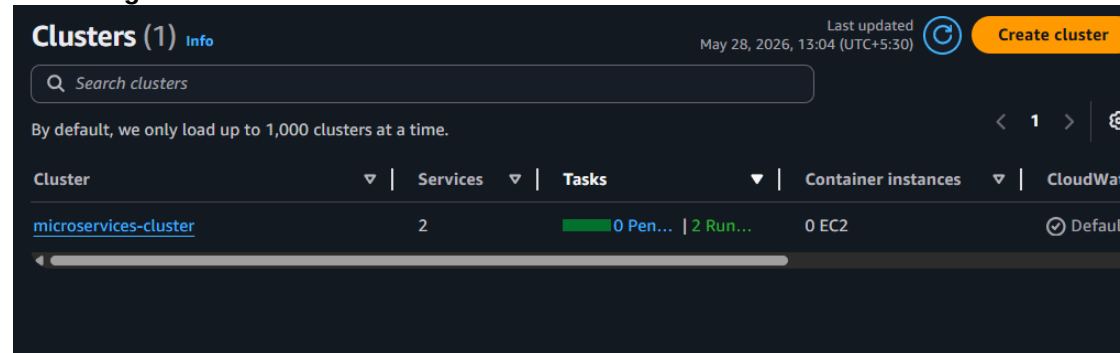
Purpose:

Make Docker images available for ECS deployment.

Step 9: Create ECS Cluster

Created:
microservices-cluster
Launch type:

AWS Fargate



Purpose:

Manage and run containerized services.

Step 10: Create Task Definitions

Created:

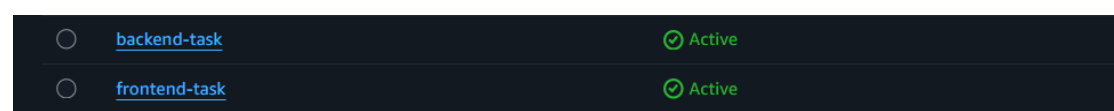
frontend-task
backend-task

Configured:

Container image
Port mapping
CPU
Memory
CloudWatch logs

Purpose:

Defines how ECS containers should run.



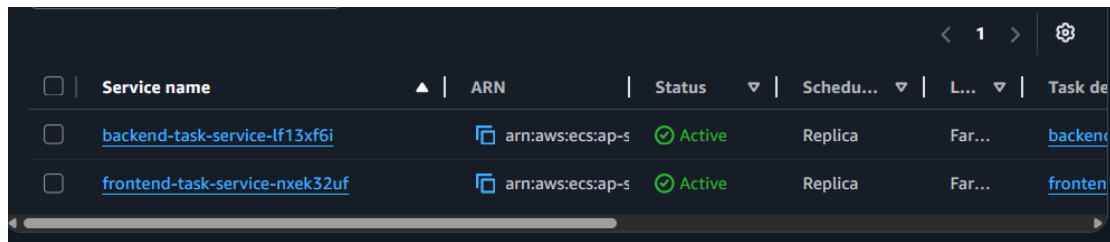
Step 11: Create ECS Services

Created:

frontend-service
backend-service

Configured:

Desired tasks = 1
Public IP enabled
Fargate launch type



☐	Service name	ARN	Status	Schedu...	L...	Task de
☐	backend-task-service-lf13xf6i	arn:aws:ecs:ap-s	Active	Replica	Far...	backen
☐	frontend-task-service-nxek32uf	arn:aws:ecs:ap-s	Active	Replica	Far...	fronten

Purpose:

Keeps containers running continuously.

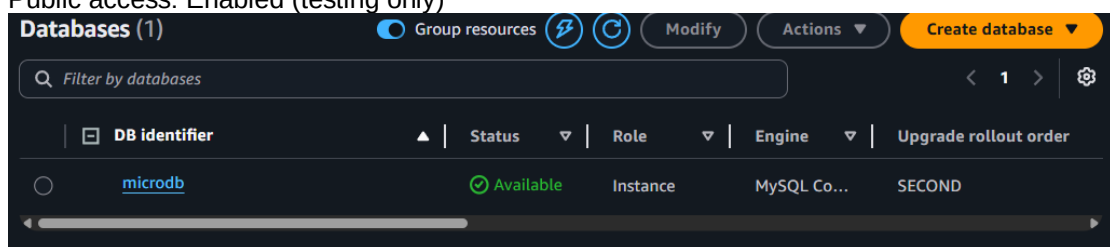
Step 12: Create Amazon RDS MySQL

Configuration:

Engine: MySQL

Database name: microdb

Public access: Enabled (testing only)



☐	DB identifier	Status	Role	Engine	Upgrade rollout order
☐	microdb	Available	Instance	MySQL Co...	SECOND

Purpose:

Store products data.

Step 13: Create Database and Table

```
yum install mariadb105 -y
```

```
mysql -h YOUR_RDS_ENDPOINT -u admin -p
```

Created database:

```
CREATE DATABASE microdb;
```

Created table:

```
CREATE TABLE products (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100),
```

```
    price INT  
);
```

Inserted data:

```
INSERT INTO products (name, price)  
VALUES ('Laptop',50000),  
      ('Phone',20000);
```

Step 14: Connect Backend with RDS

Updated backend database connection:

```
const db = mysql.createConnection({  
  host: "RDS-ENDPOINT",  
  user: "admin",  
  password: "PASSWORD",  
  database: "microdb",  
});
```

Purpose:

Backend fetches products from RDS instead of hardcoded JSON.

Step 15: CloudWatch Logs

Enabled CloudWatch logs inside ECS task definitions.

Purpose:

- Monitor application logs
- Debug container issues
- Track deployment errors

Security Group Configuration

1. Frontend Security Group

Inbound rule:

Type: Custom TCP
Port: 5173
Source: 0.0.0.0/0

Purpose:

Allows users to open frontend in browser.

2. Backend Security Group

Inbound rule:

Type: Custom TCP
Port: 5000
Source: frontend-sg

Purpose:

Only frontend can call backend API.

3. RDS Security Group

Inbound rule:

Type: MySQL/Aurora
Port: 3306
Source: backend-sg

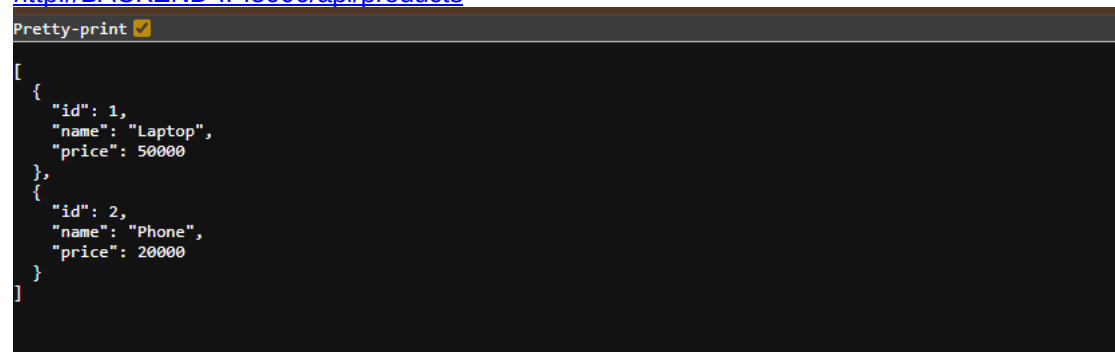
Purpose:

Only backend can connect to database.

Testing

Backend API Test

<http://BACKEND-IP:5000/api/products>

A screenshot of a web browser window. The address bar shows the URL 'http://BACKEND-IP:5000/api/products'. The page content displays a JSON array of two objects, each representing a product. The first object has 'id': 1, 'name': 'Laptop', and 'price': 50000. The second object has 'id': 2, 'name': 'Phone', and 'price': 20000. A 'Pretty-print' button with a checkmark icon is visible in the top right corner of the response area.

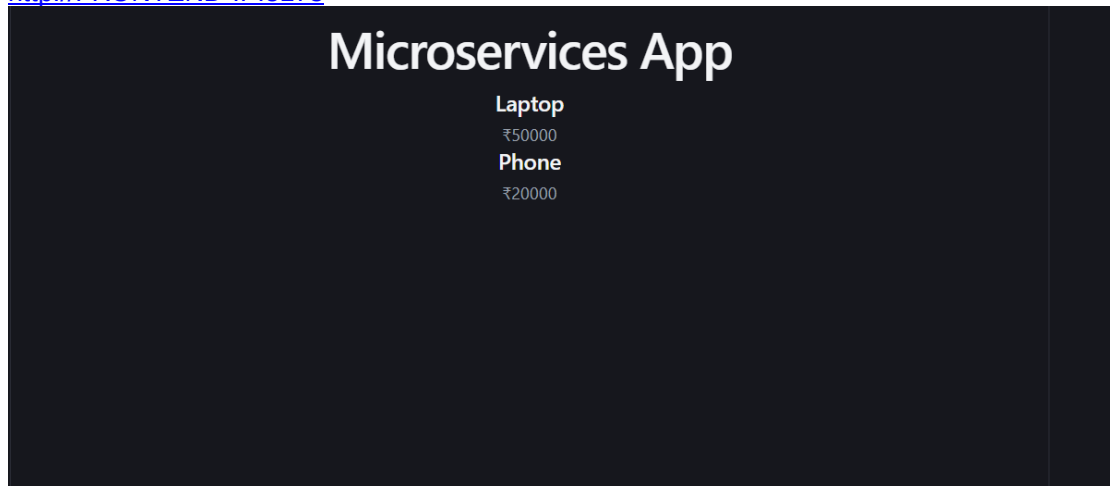
```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 50000
  },
  {
    "id": 2,
    "name": "Phone",
    "price": 20000
  }
]
```

Expected:

```
[
  {
    "id":1,
    "name":"Laptop",
    "price":50000
  }
]
```

Frontend Test

<http://FRONTEND-IP:5173>



Expected:

Products displayed from database.

Troubleshooting

Issue 1: Docker Compose Not Found

Solution

Installed Docker Compose manually.

Issue 2: ECS Timeout

Solution

Added correct inbound security group rules.

Issue 3: RDS Connection Timeout

Solution

Configured correct RDS security group rules.

Issue 4: Old ECS Image Running

Solution

Used:

Force new deployment

inside ECS service.

Issue 5: Frontend Not Showing Data

Solution

Updated frontend API URL to ECS backend public IP.

Conclusion

This project successfully deploys a scalable microservices application using AWS ECS and Fargate.

The application was containerized using Docker, images were stored inside Amazon ECR, services were deployed serverlessly using ECS Fargate, backend APIs connected with Amazon RDS MySQL, and CloudWatch was used for monitoring and logs.

This project demonstrates:

- Containerization
- Microservices architecture
- Serverless containers
- Cloud deployment
- AWS networking
- Database integration
- Production-style architecture
- Security best practices
- Monitoring and troubleshooting